MODULE

11

Advanced Git

Hooks, worktrees, submodules, line endings, secrets in history, LFS and huge repos.

YOU WILL LEARN

Hooks	`01_hooks.py`
Worktrees	`02_worktrees.py`
Submodules & subtree	`03_submodules_and_subtree.py`
Line endings	`04_line_endings_and_attributes.py`
Secrets, LFS, signing	`05_secrets_big_files_and_signing.md`
Exercise: secret guard hook	`ex11_secret_guard_hook.py`
Interview Q&A	`INTERVIEW.md`

WHY ADVANCED GIT

Problems that appear in real teams

THE PROBLEM

An API key pushed to GitHub. A diff where all 500 lines ”changed”. A hotfix needed while your dev server is running. A 3 GB repository nobody can clone.

THE IDEA

Each has a precise tool: hooks, .gitattributes, worktrees, filter-repo, LFS, partial clones.

TRY IT HOOKS

A pre-commit hook in 5 lines

```
#!/bin/sh
if git diff --cached | grep -qE "(PASSWORD|SECRET|API_KEY)="; then
  echo "pre-commit: a secret is staged. Commit blocked."
  exit 1
fi
```

1. Lives in **.git/hooks/pre-commit** (make it executable).
2. Exit 1 = the commit is blocked.
3. Others: **commit-msg**, **pre-push**; share with pre-commit.com.

TRY IT HOOKS

The hook in action

```
$ echo "API_KEY=sk_live_123" > settings.py
$ git add settings.py
$ git commit -m "Add settings"
pre-commit: a secret is staged. Commit blocked.
$ git log --oneline
3d365c0 Initial commit
```

1. The commit never happened.
2. Bypass: **--no-verify**, so CI must run the same checks.
3. Real teams: ruff, gitleaks, detect-secrets.

TRY IT WORKTREES

Two branches open at the same time

```
$ git worktree add ../shop-hotfix hotfix/rounding
HEAD is now at dd073e8 Add cart
Preparing worktree (checking out 'hotfix/rounding')
$ git worktree list
~/project     dd073e8 [feature/coupons]
~/shop-hotfix dd073e8 [hotfix/rounding]
$ git status --short
?? coupon.py
```

1. A second folder sharing the same .git.
2. Your unfinished feature stays untouched.
3. **git worktree remove** when done.

STEP BY STEP REUSING CODE

Submodule vs subtree vs package

	SUBMODULE	SUBTREE	PACKAGE (PIP/NPM)
What's in your repo	a pointer to a commit	a copy of the files	a version number
Clone needs	`--recurse-submodules`	nothing special	pip install
Update	`git submodule update --remote`	`git subtree pull`	bump the version
Best for	pinned infra modules, protos	vendoring small libs	most shared libraries

TRY IT LINE ENDINGS

Fix CRLF vs LF once for the whole team

```
$ git ls-files --eol
i/crlf  w/crlf  attr/text=auto eol=lf   deploy.sh
$ cat .gitattributes
* text=auto eol=lf
*.bat text eol=crlf
*.png binary
$ git add --renormalize .
$ git add .gitattributes
$ git commit -qm "Normalise line endings"
$ git ls-files --eol
i/lf    w/lf    attr/text=auto eol=lf   .gitattributes
i/lf    w/crlf  attr/text=auto eol=lf   deploy.sh
```

1. **i/crlf**: the repo stored Windows endings.
2. **.gitattributes** is the team rule.
3. After renormalising: **i/lf**.

COMMON MISTAKE SECRETS

”Fixing” a leaked key with a new commit

✗ WRONG

```
# pushed: API_KEY=sk_live_123
git rm --cached .env
git commit -m "Remove secret"
git push
# the key is still in history (and already scraped)
```

Anyone can read the old commit. Bots find public keys within minutes. The key is compromised.

✓ RIGHT

```
# 1. ROTATE the key now (provider dashboard)
# 2. clean history:
git filter-repo --replace-text expressions.txt
git push --force --all
# 3. prevent: .gitignore, gitleaks, push protection
```

Rotate first. Then clean history and make everyone re-clone.

STEP BY STEP BIG REPOS

Big files and huge repositories

PROBLEM	TOOL
Large binaries (videos, models, PSDs)	**Git LFS**: `git lfs track "*.mp4"`
CI only needs the latest code	`git clone --depth 1` (shallow)
History yes, all blobs no	`git clone --filter=blob:none` (partial)
Only some folders of a monorepo	`git sparse-checkout set services/api`
Prove who made a commit	signed commits (SSH/GPG) → **Verified**

IN REAL JOBS ADVANCED GIT

Senior-level habits

Shared hooks

```
pip install pre-commit
pre-commit install    # .pre-commit-config.yaml
```

Day-one repo hygiene

```
.gitignore + .gitattributes + CODEOWNERS
```

Fast CI checkouts

```
git clone --depth 1 --filter=blob:none <url>
```

Signed commits

```
git config --global gpg.format ssh
git config --global commit.gpgsign true
```

INTERVIEW SAY THIS IN THE INTERVIEW

“You pushed a password to GitHub. What do you do?”

- **Rotate the credential immediately**: assume it’s compromised.
- Not pushed yet? Amend or rebase it out.
- Pushed: **git filter-repo** to remove it from all history, force-push, everyone re-clones.
- Prevent: .gitignore, pre-commit secret scanning, GitHub **push protection**, a vault for secrets.

> **Follow-up they ask next:** ”What are Git hooks?” (scripts on events like pre-commit/commit-msg/pre-push; share via pre-commit; CI repeats them).

RECAP REMEMBER THESE

Module 11 in 5 lines

1. **Hooks** guard commits locally; CI repeats the checks.
2. **worktree**: another branch in another folder, same repo.
3. **Submodules** pin other repos; packages are often simpler.
4. **.gitattributes** + **--renormalize** end line-ending wars.
5. Leaked secret: rotate, filter-repo, prevent. Big files: LFS.

PRACTICE NOW

Do exercises/ex11_secret_guard_hook.py: block secrets with a hook.